PH 724 Data Analysis & Scientific Communication Incorporating Use of Generative AI

Part 8: Introduction to Splines

Zhiyang Zhou (zhou67@uwm.edu, zhiyanggeezhou.github.io)

2026/10/27 06:07:26

Polynomials

- A polynomial is a function consisting of a sum of powers of a variable t, t^2, t^3, etc., multiplied by coefficients:
  - Linear polynomial (degree 1): $f(t) = \beta_0 + \beta_1 t$
  - Quadratic polynomial (degree 2): $f(t) = \beta_0 + \beta_1 t + \beta_2 t^2$
  - Cubic polynomial (degree 3): $f(t) = \beta_0 + \beta_1 t + \beta_2 t^2 + \beta_3 t^3$
- The degree is the highest power of t with a nonzero coefficient.
- A spline is a smooth curve constructed by joining simpler polynomial pieces, using **different polynomial pieces in different regions**.

From polynomials to splines

- A single polynomial may be too restrictive to represent a complicated relationship.
- Idea: Divide the domain into several intervals and fit a polynomial piece within each interval.
- The locations where the polynomial pieces meet are called **knots**.
- The resulting piecewise polynomial function is called a **spline**, provided the pieces satisfy the specified continuity conditions.
- For example:
  - Linear spline: joins linear pieces continuously.
  - Quadratic spline: joins quadratic pieces with continuous first derivatives.
  - Cubic spline: joins cubic pieces with continuous first and second derivatives.
- Splines are commonly used to approximate unknown functions.
  - Splines can approximate smooth functions with increasingly small error as the spacing between knots decreases.

Fitting splines to Abrabidopsis data

```
library(ggplot2)
library(splines)

# Read the original data from McCullagh's textbook
```

```r
url <- "https://www.stat.uchicago.edu/~pmcc/projects/ch04/PlantGrowth.dat"
raw <- read.table(url, header = FALSE)

# Columns: plant ID, strain, 11 height measurements
names(raw) <- c(
  "plant", "strain",
  paste0("day", seq(29, 69, by = 4))
)

# Select strain cis (strain = 0)
cis <- subset(raw, strain == 0)

# Convert to long format
dates <- seq(29, 69, by = 4)

d <- data.frame(
  plant = rep(cis$plant, each = length(dates)),
  date = rep(dates, times = nrow(cis)),
  height = as.vector(t(as.matrix(cis[, -(1:2)])))
)

# Plant birth = last date with zero height
birth <- aggregate(
  date ~ plant,
  data = subset(d, height == 0),
  FUN = max
)
names(birth)[2] <- "birth"

d <- merge(d, birth, by = "plant")
d$age <- d$date - d$birth
d <- subset(d, age >= 0)

# Mean observed height at each age
d_mean <- aggregate(height ~ age, data = d, FUN = mean)

# Same knots for all three splines
knots <- c(10, 20, 30)

degrees <- c(
  "Linear spline" = 1,
  "Quadratic spline" = 2,
  "Cubic spline" = 3
)

# Fit each spline directly to individual observations
fits <- lapply(degrees, function(deg) {
  lm(
    height ~ bs(
      age,
      knots = knots,
      degree = deg,
      Boundary.knots = c(0, 40)
```

```
    ),
    data = d
  )
})

# Predicted population mean curves
grid <- data.frame(
  age = seq(0, 40, length.out = 401)
)

curve <- do.call(
  rbind,
  lapply(names(fits), function(type) {
    data.frame(
      age = grid$age,
      height = predict(fits[[type]], newdata = grid),
      spline = type
    )
  })
)

# Highlight fitted values at knots
knot_points <- do.call(
  rbind,
  lapply(names(fits), function(type) {
    data.frame(
      age = knots,
      height = predict(
        fits[[type]],
        newdata = data.frame(age = knots)
      ),
      spline = type
    )
  })
)

# Repeat observations across the three panels
d_plot <- do.call(
  rbind,
  lapply(names(degrees), function(type) {
    data.frame(
      d,
      spline = type
    )
  })
)

curve$spline <- factor(curve$spline, levels = names(degrees))
knot_points$spline <- factor(
  knot_points$spline,
  levels = names(degrees)
)
d_plot$spline <- factor(d_plot$spline, levels = names(degrees))
```

```r
ggplot() +
  geom_point(
    data = d_plot,
    aes(age, height),
    color = "#7a9fbd",
    alpha = 0.25,
    size = 1.5,
    position = position_jitter(
      width = 0.25,
      height = 0
    )
  ) +
  geom_vline(
    xintercept = knots,
    linetype = "dashed",
    color = "gray55"
  ) +
  geom_line(
    data = curve,
    aes(age, height),
    color = "#b4443b",
    linewidth = 1.3
  ) +
  geom_point(
    data = knot_points,
    aes(age, height),
    shape = 21,
    fill = "white",
    color = "#b4443b",
    size = 3.5,
    stroke = 1.2
  ) +
  facet_wrap(~ spline, nrow = 1) +
  labs(
    x = "Plant age (days)",
    y = "Plant height (mm)",
    subtitle = "Fitted linear/quadratic/cubic splines Arabidopsis strain cis",
    caption = paste(
      "Blue: individual observations;",
      "red: fitted mean growth curves;",
      "dashed lines: knots;",
      "open circles: fitted values at knots."
    )
  ) +
  theme_minimal(base_size = 16) +
  theme(
    strip.text = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )
```

B-splines: Facilitating the construction of splines

- A spline can be expressed as a weighted sum of **basis splines** (B-splines). Smooth unknown functions hence can be approximated as a linear combination of B-splines:

$$f(t) = \sum_{k=1}^{K} \beta_k B_k(t).$$

- $B_k(t)$: a known B-spline, determined by the knots, degree and boundary conditions. Its math expression is complicated, but it can be computed using the `splines::bs()` function in `R`.
- β_k: an unknown coefficient estimated from the data.
  - Once B-splines are selected, estimating the coefficients can be treated as a regression problem.
- K: the degree of freedom.
- **Intuition:** Each B-spline contributes part of the final curve. By adjusting their coefficients, we obtain different shapes.

Visualizing B-splines

- Each B-spline is active over a limited interval.
- Neighboring B-splines overlap.

```
library(ggplot2)
library(splines)

t <- seq(0, 40, length.out = 300)

# Cubic B-splines of degree 3 with 6 degrees of freedom
B <- bs(t, df = 6, degree = 3, intercept = TRUE)

d <- data.frame(
  time = rep(t, ncol(B)),
  value = as.vector(B),
  basis = factor(
    rep(seq_len(ncol(B)), each = length(t))
  )
)

ggplot(d, aes(time, value, color = basis)) +
  geom_line(linewidth = 1.2) +
  labs(
    x = "Time",
    y = "Basis function value",
    subtitle = paste("B-splines of degree", attr(B, "degree"), "with", ncol(B), "degrees of freedom"),
    color = "Basis"
  ) +
  theme_minimal(base_size = 16)
```

Fitting B-splines to plant-growth observations

```
library(ggplot2)
library(splines)
```

```r
# Read the original data from McCullagh's textbook
url <- "https://www.stat.uchicago.edu/~pmcc/projects/ch04/PlantGrowth.dat"
raw <- read.table(url, header = FALSE)

# Columms: plant ID, strain, 11 height measurements
names(raw) <- c(
  "plant", "strain",
  paste0("day", seq(29, 69, by = 4))
)

# Select strain cis (strain = 0)
cis <- subset(raw, strain == 0)

# Convert to long format
dates <- seq(29, 69, by = 4)

d <- data.frame(
  plant = rep(cis$plant, each = length(dates)),
  date = rep(dates, times = nrow(cis)),
  height = as.vector(t(as.matrix(cis[, -(1:2)])))
)

# Plant birth = last date with zero height
birth <- aggregate(
  date ~ plant,
  data = subset(d, height == 0),
  FUN = max
)
names(birth)[2] <- "birth"

d <- merge(d, birth, by = "plant")
d$age <- d$date - d$birth
d <- subset(d, age >= 0)

# Mean observed height at each age
d_mean <- aggregate(height ~ age, data = d, FUN = mean)

# Same knots for all three splines
knots <- c(10, 20, 30)

degrees <- c(
  "Linear spline" = 1,
  "Quadratic spline" = 2,
  "Cubic spline" = 3
)

# Fit each spline directly to individual observations
degree_bs = 3
fit <- lm(
  height ~ bs(
    age,
    knots = c(10, 20, 30),
    degree = degree_bs,
```

```
    Boundary.knots = c(0, 40)
  ),
  data = d
)

grid <- data.frame(age = seq(0, 40, length.out = 401))
grid$height <- predict(fit, newdata = grid)

ggplot() +
  geom_point(
    data = d,
    aes(age, height),
    color = "#7a9fbd",
    alpha = 0.25,
    position = position_jitter(
      width = 0.25,
      height = 0,
      seed = 724
    )
  ) +
  geom_vline(
    xintercept = c(10, 20, 30),
    linetype = "dashed",
    color = "gray55"
  ) +
  geom_line(
    data = grid,
    aes(age, height),
    color = "#b4443b",
    linewidth = 1.3
  ) +
  labs(
    x = "Plant age (days)",
    y = "Plant height (mm)",
    subtitle = paste("B-spline of degree", degree_bs, "fit to the cis strain")
  ) +
  theme_minimal(base_size = 16)
```

- The regression illustrates the mean relationship; it does not explicitly account for correlation among repeated measurements from the same plant.

From B-splines to I-splines

- B-splines does NOT guarantee that the fitted function is increasing.
- **M-splines** are normalized B-splines such that

$$\int M_k(t)\, dt = 1$$

- An **I-spline** is defined as the integral of an M-spline:

$$I_k(t) = \int_a^t M_k(u)\, du$$

where a is the left boundary of the spline domain.

- Because $M_k(t) \geq 0$, $I_k'(t) = M_k(t) \geq 0$. That is, each I-spline is **nondecreasing**.
- We can the approximatet an non-decreasing function as a linear combination of I-splines:

$$f(t) = \beta_0 + \sum_{k=1}^{K} \beta_k I_k(t), \qquad \beta_k \geq 0.$$

- Any continuous nondecreasing function on a compact interval may be approximated arbitrarily closely by a linear combination of I-splines, provided that the number of knots is sufficiently large.

Fitting I-splines to plant-growth observations

```
library(ggplot2)
library(splines2)
library(nnls)

# Original data from the textbook
url <- "https://www.stat.uchicago.edu/~pmcc/projects/ch04/PlantGrowth.dat"
raw <- read.table(url, header = FALSE)

# Columns: plant ID, strain, 11 height measurements
names(raw) <- c(
  "plant", "strain",
  paste0("day", seq(29, 69, by = 4))
)

# Select strain cis (strain = 0)
cis <- subset(raw, strain == 0)

# Convert to long format
dates <- seq(29, 69, by = 4)

d <- data.frame(
  plant = rep(cis$plant, each = length(dates)),
  date = rep(dates, times = nrow(cis)),
  height = as.vector(t(as.matrix(cis[, -(1:2)])))
)

# Plant birth = last date with zero height
birth <- aggregate(
  date ~ plant,
  data = subset(d, height == 0),
  FUN = max
)

names(birth)[2] <- "birth"

d <- merge(d, birth, by = "plant")
d$age <- d$date - d$birth
d <- subset(d, age >= 0)

# Interior knots and boundary knots
knots <- c(10, 20, 30)
bounds <- c(0, 40)
```

```r
# Construct cubic I-spline basis
I <- iSpline(
  d$age,
  knots = knots,
  degree = 3,
  Boundary.knots = bounds,
  intercept = TRUE
)

# Nonnegative least squares:
# E(height | age) = sum(theta_k * I_k(age)), theta_k >= 0
# No intercept: fitted mean height is zero at age 0
fit <- nnls(I, d$height)

theta <- coef(fit)

# Prediction grid
grid <- data.frame(
  age = seq(0, 40, length.out = 401)
)

I_grid <- iSpline(
  grid$age,
  knots = knots,
  degree = 3,
  Boundary.knots = bounds,
  intercept = TRUE
)

grid$height <- as.vector(I_grid %*% theta)

# Fitted values at knots
knot_points <- data.frame(age = knots)

I_knots <- iSpline(
  knot_points$age,
  knots = knots,
  degree = 3,
  Boundary.knots = bounds,
  intercept = TRUE
)

knot_points$height <- as.vector(I_knots %*% theta)

# Plot
ggplot() +
  geom_point(
    data = d,
    aes(age, height),
    color = "#7a9fbd",
    alpha = 0.25,
    size = 1.5,
    position = position_jitter(
```

```
      width = 0.25,
      height = 0,
      seed = 724
    )
  ) +
  geom_vline(
    xintercept = knots,
    linetype = "dashed",
    color = "gray55"
  ) +
  geom_line(
    data = grid,
    aes(age, height),
    color = "#b4443b",
    linewidth = 1.3
  ) +
  geom_point(
    data = knot_points,
    aes(age, height),
    shape = 21,
    fill = "white",
    color = "#b4443b",
    size = 3.5,
    stroke = 1.2
  ) +
  labs(
    x = "Plant age (days)",
    y = "Plant height (mm)",
    subtitle = "Cubic I-spline fit: Arabidopsis strain cis",
    caption = paste(
      "Blue: individual observations;",
      "red: monotone fitted mean growth curve;",
      "dashed lines: knots;",
      "open circles: fitted values at knots."
    )
  ) +
  theme_minimal(base_size = 16) +
  theme(
    panel.grid.minor = element_blank()
  )
```

Choosing the number and locations of knots

- The (polynomial) degree controls smoothness at the knots, while the number and placement of knots control the flexibility of the fitted curve.
- Too few knots: The fitted curve may be too rigid to capture the true growth pattern.
- Too many knots: The fitted curve may follow random noise rather than the underlying relationship, leading to overfitting.
- Knots can be placed at equally spaced ages or at quantiles of the observed times.
- Smoothing splines may avoid the need to choose the number and locations of knots by using a large number of knots and penalizing the roughness of the fitted curve.

Smoothing splines

- Intuition: A smoothing spline balances two competing goals:
  - Fit the observed data well.
  - Avoid an unnecessarily wiggly curve.
- A smoothing spline minimizes

$$\sum_{i=1}^{n}\{Y_i - f(t_i)\}^2 + \lambda \int \{f''(t)\}^2 \mathrm{dt}$$

  - $f(t)$ is the fitted spline, typically expressed as a linear combination of basis functions; e.g., $f(t) = \sum_{k=1}^{K} \beta_k B_k(t)$, or $f(t) = \beta_0 + \sum_{k=1}^{K} \beta_k I_k(t)$.
  - $f''(t)$ is the second derivative of $f(t)$, which measures the curvature of the fitted curve.
  - $\lambda \geq 0$ is the smoothing parameter.
    - When $\lambda = 0$, the smoothing spline reduces to the **regression spline**.
    - A small λ allows a more flexible curve.
    - A large λ favors a smoother curve; in the limiting case, the fit approaches a straight line.
    - λ can be selected using methods such as cross-validation or generalized cross-validation.

What happens when we change the smoothing parameter?

- Penalization can prevent a model with many basis functions from becoming excessively wiggly.
- This illustrates the bias–variance tradeoff:
  - A very restrictive model may have substantial bias.
  - An overly flexible model may have high estimation variance.
- The objective is not to fit every observation perfectly, but to estimate the underlying relationship reliably.

```
library(ggplot2)
set.seed(724)

t <- seq(0, 40, length.out = 45)

true_mean <- 30 * t^2 / (12^2 + t^2)
y <- true_mean + rnorm(length(t), sd = 1.5)

obs <- data.frame(time = t, height = y)

# Smoothing splines with different smoothing parameters
fit_low  <- smooth.spline(t, y, lambda = 0.001)
fit_mid  <- smooth.spline(t, y, lambda = 0.1)
fit_high <- smooth.spline(t, y, lambda = 10)

grid <- seq(0, 40, length.out = 300)

pred <- data.frame(
  time = rep(grid, 3),
  height = c(
    predict(fit_low, grid)$y,
    predict(fit_mid, grid)$y,
    predict(fit_high, grid)$y
  ),
  smoothing = factor(
    rep(
```

```
      c("lambda = 0.001", "lambda = 0.1", "lambda = 10"),
      each = length(grid)
    ),
    levels = c("lambda = 0.001", "lambda = 0.1", "lambda = 10")
  )
)

ggplot() +
  geom_point(
    data = obs,
    aes(x = time, y = height),
    color = "#7a9fbd",
    alpha = 0.6
  ) +
  geom_line(
    data = pred,
    aes(x = time, y = height),
    color = "#b4443b",
    linewidth = 1.2
  ) +
  facet_wrap(~ smoothing, nrow = 1) +
  labs(
    x = "Time (days)",
    y = "Plant height (mm)",
    subtitle = "The smoothing parameter controls the flexibility of the fitted curve"
  ) +
  theme_minimal(base_size = 16)
```

Real-world applications of splines

- Temperature and mortality in New York City. Gasparrini and colleagues analyzed daily mortality in New York City from 1987–2000. They used natural cubic spline bases to model how temperature affects mortality and how the effect changes over subsequent days. See https://pmc.ncbi.nlm.nih.gov/articles/PMC2998707/.
- Temperature and mortality across England and Wales. A study of 10 regions in England and Wales used quadratic B-splines for temperature and natural cubic B-splines for lag. The resulting model describes the joint temperature-lag-mortality relationship. See https://pmc.ncbi.nlm.nih.gov/articles/PMC3599933/.
- Drug dose-response and interactions. Wan and colleagues used penalized I-splines with coefficient constraints to estimate monotone dose-response relationships. See https://pmc.ncbi.nlm.nih.gov/articles/PMC5217167/.